Project Report: Smart Parking Management System

1. Introduction

The **Smart Parking Management System** (ParkSystem) is an enterprise-grade, real-time web application designed to streamline the parking experience for organizations, facilities, and users. The system leverages modern web technologies to provide role-based access control, real-time telemetry updates, and seamless reservation workflows.

2. Objectives

- **Centralized Management**: To provide facility managers and administrators a bird's-eye view of parking slot occupancy and reservations.
- **Real-Time Data Sync**: To ensure all users see accurate availability data instantaneously without refreshing the page.
- **Role-Based Access**: To isolate functionality based on user roles (Super Admin, Facility Manager, Parking Administrator, Security Officer, Employee, Visitor).
- **Automated Workflows**: To handle the complete lifecycle of a reservation (Pending, Confirmed, Checked-In, Completed) including automated QR code generation and email notifications.

3. Technology Stack

The project adopts a modern full-stack JavaScript ecosystem:

- **Frontend**: React 18, Vite (for ultra-fast bundling), Tailwind CSS (for responsive, utility-first styling).
- **Backend**: Node.js, Express.js (RESTful API architecture).
- **Real-Time Communication**: Socket.IO (using targeted rooms for scoped event broadcasting).
- **Database**: SQLite (via `better-sqlite3` for fast, synchronous read/write operations).
- **Authentication**: JSON Web Tokens (JWT) and `bcryptjs` for secure password hashing.
- **Notifications**: Nodemailer (Ethereal SMTP) and `qrcode` for automated email delivery.

4. System Architecture

The application is structured into two primary directories interacting over HTTP and WebSockets:

4.1 Frontend Architecture (`smart_parking_react`)

The frontend is built using React Context API for state management:

- `AuthContext`: Handles JWT token lifecycle, login/logout, and provides a customized `authFetch` wrapper for authenticated API calls.
- `SocketContext`: Manages the Socket.IO client instance, emitting an `authenticate` event on connection to join appropriate targeted rooms.
- `ToastContext`: Provides a unified notification queue for non-blocking user feedback.

4.2 Backend Architecture (`backend`)

The backend is a monolithic Express.js server (`server.js`):

- **Dynamic Routing**: A centralized `tableEventMap` dynamically generates CRUD routes (`GET`, `POST`, `PUT`, `DELETE`) for all database tables, automatically triggering real-time websocket broadcasts upon data mutation.
- **Data Layer**: The `db.js` file handles SQLite table initialization and database seeding for rapid development and testing.
- **Security**: Routes are protected by an `authenticate` JWT middleware, and login endpoints are guarded by `express-rate-limit` to prevent brute-force attacks.

5. Key Features & Implementation Details

5.1 Real-Time Targeted Updates

Instead of broadcasting every database change to all connected clients, the system uses **Socket.IO Rooms**. Upon login, the client emits its `userId` and `role`.

- Visitors are joined to a room specific to their `userId` (e.g., `user_6`).
- Administrators are joined to a global `admin` room.
  When a reservation is updated, the backend selectively emits the event to `user_6` and `admin`, drastically reducing unnecessary network traffic for other active users.

5.2 Granular Reservation State Machine

The core business logic centers around the reservation lifecycle:

1. **Pending**: A Visitor or Employee requests a parking slot.
2. **Confirmed**: An Administrator approves the request.
3. **Checked-In**: The user physically arrives at the facility (can be triggered by a Security Officer).
4. **Completed / No-Show**: The terminal state of the reservation.

5.3 Automated Email & QR Pipeline

When a reservation transitions from `Pending` to `Confirmed`, the backend intercepts the PUT request. It invokes the `emailService`, generating a unique QR code (containing `PARK_RES_{ID}_{DATE}`) and embedding it into an HTML email template. This is simulated using Nodemailer Ethereal.

6. Division of Responsibilities (Team Structure)

The project is modularly structured, allowing it to be divided among a team of three:

1. **Frontend Engineer**: Responsible for the React UI, Tailwind CSS styling, Client-side routing, and Context API implementation.
2. **Backend & Real-Time Engineer**: Responsible for Node.js architecture, Socket.IO real-time rooms, JWT security, and email notification pipelines.
3. **Database & Systems Engineer**: Responsible for SQLite schema design, dynamic RESTful API generation, and complex state machine business logic.

7. Future Enhancements

To scale this application for a live enterprise environment, the following enhancements are proposed:

- **Database Migration**: Upgrading from SQLite to PostgreSQL to handle high-concurrency write operations.
- **Hardware IoT Integration**: Exposing webhooks for License Plate Recognition (LPR) cameras or boom barriers to automate the "Check-In" status.
- **Payment Gateway**: Integrating Stripe API for paid visitor parking reservations.
- **Containerization**: Implementing Docker for seamless deployment across cloud providers.

8. Conclusion

The Smart Parking Management System successfully demonstrates a robust, real-time web application. By combining modern React paradigms with an event-driven Node.js backend, the system solves real-world facility management challenges while adhering to software engineering best practices such as RBAC, targeted websocket broadcasting, and modular architecture.

9. Detailed Workflows & Page Features

The application provides a seamless, unified workflow across multiple customized dashboards. Below is a detailed breakdown of each page, the features available, and the resulting actions upon user interaction.

9.1 Authentication & Global Layout

- **Login Page (`/authentication-login`)**
    - **Features**: User authentication via email and password.
    - **Action & Workflow**: Upon submitting valid credentials, the backend generates a JWT. The frontend stores this token and redirects the user to their designated dashboard based on their role (e.g., `superadmin` goes to Admin Dashboard, `visitor` goes to Visitor Dashboard).
- **Global Layout & Navigation**
    - **Features**: Sidebar navigation, Global Search, Notification Bell, Dark/Light Mode toggle, User Profile dropdown.
    - **Action & Workflow**:
        - *Sidebar Toggle*: Clicking the hamburger menu collapses/expands the navigation sidebar.
        - *Theme Toggle*: Clicking the sun/moon icon instantly swaps the global CSS variables between light and dark themes.
        - *Notification Bell*: Clicking the bell fetches the latest role-specific unread alerts. Clicking "View All" navigates to the Notification Center.

9.2 Role-Based Dashboards

- **Admin Dashboard (`/admin-dashboard`)**
    - **Access**: Super Admin, Facility Manager.
    - **Features**: Live KPI metric cards (Total Capacity, Occupancy %, Active Sessions), quick action buttons, and a live parking map preview.
    - **Action & Workflow**: Clicking a "Quick Action" (like overriding a slot or approving a reservation) immediately dispatches an API request and broadcasts a WebSocket update.
- **Employee Dashboard (`/employee-dashboard`)**
    - **Access**: Super Admin, Employee.
    - **Features**: Personalized overview of upcoming reservations and assigned vehicles.
    - **Action & Workflow**: Employees can click "Book Parking" to navigate directly to the Reservation Module with their ID pre-filled.
- **Visitor Dashboard (`/landing-dashboard`)**
    - **Access**: Super Admin, Visitor.
    - **Features**: Simplified interface showing current reservation status (Pending, Confirmed, Checked-In) and a button to request a new slot.
    - **Action & Workflow**: Clicking "Request Slot" opens a modal to choose a date, time, and slot. Submitting creates a `Pending` reservation and notifies administrators.

9.3 Core Management Modules

- **Slot Management (`/slot-management`)**
    - **Access**: Super Admin, Parking Administrator.
    - **Features**: List view of all physical parking spaces, including level, type (EV, VIP, Standard), and current status.
    - **Action & Workflow**: Administrators can click "Edit" to mark a slot as `Maintenance`. This updates the database, re-calculates the live capacity metric, and pushes a real-time update to all dashboards.
- **Vehicle Registry (`/vehicle-management`)**
    - **Access**: Super Admin, Facility Manager, Parking Administrator, Employee.
    - **Features**: CRUD interface for registering license plates, vehicle make/model, and assigning defaults.
    - **Action & Workflow**: Clicking "Add Vehicle" registers a new car to the user's profile. Administrators can click "Verify" to approve a vehicle for automatic gate access.
- **Visitor Access (`/visitor-management`)**
    - **Access**: Super Admin, Facility Manager, Security Officer.
    - **Features**: Real-time log of expected and arrived physical visitors.
    - **Action & Workflow**: A Security Officer can click "Mark Arrived" when a visitor pulls up to the gate. This updates the visitor's status and automatically sends a real-time notification to the host employee.

9.4 Reservation Lifecycle & Real-Time Map

- **Reservation Module (`/reservation-module`)**
    - **Access**: Super Admin, Parking Administrator, Employee, Visitor.
    - **Features**: Comprehensive table of all bookings. Visitors see their own; Admins see all.
    - **Action & Workflow**:
        1. **Create**: User clicks "New Reservation" -> Creates a `Pending` request.
        2. **Approve**: Admin clicks "Approve" -> Status changes to `Confirmed`. The backend generates a QR code and sends an automated confirmation

email to the user.
3. **Check-In**: Security clicks "Check-In" -> Status changes to `Checked-In`, and the corresponding parking slot is marked as `Occupied`.
4. **Complete**: Admin clicks "Complete" -> Slot is freed up, and metrics recalculate.

- **Live Parking Map (`/live-parking-map`)**
  - **Access**: Super Admin, Parking Administrator, Security Officer.
  - **Features**: Visual grid representation of the facility. Slots are color-coded (Green = Available, Red = Occupied, Yellow = Reserved, Gray = Maintenance).
  - **Action & Workflow**: The map listens to Socket.IO events. If a reservation is approved or a slot is taken on another screen, the map tile changes color instantly without a page refresh.

9.5 Analytics & User Administration

- **Reports & Analytics (`/reports-analytics`)**
  - **Access**: Super Admin, Facility Manager.
  - **Features**: Charts (e.g., peak occupancy times, vehicle type distribution) and tabular data.
  - **Action & Workflow**: Clicking "Export to CSV" (Future Feature) compiles the historical data table into a downloadable spreadsheet.
- **User Management (`/user-management`)**
  - **Access**: Super Admin only.
  - **Features**: List of all system users, their roles, and account statuses.
  - **Action & Workflow**: Clicking "Deactivate" immediately revokes the user's access. The next time the deactivated user attempts an API call, their JWT token is rejected.